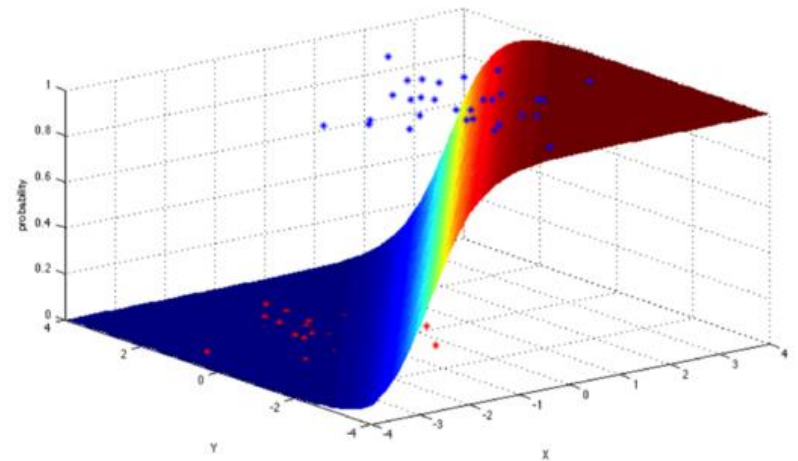
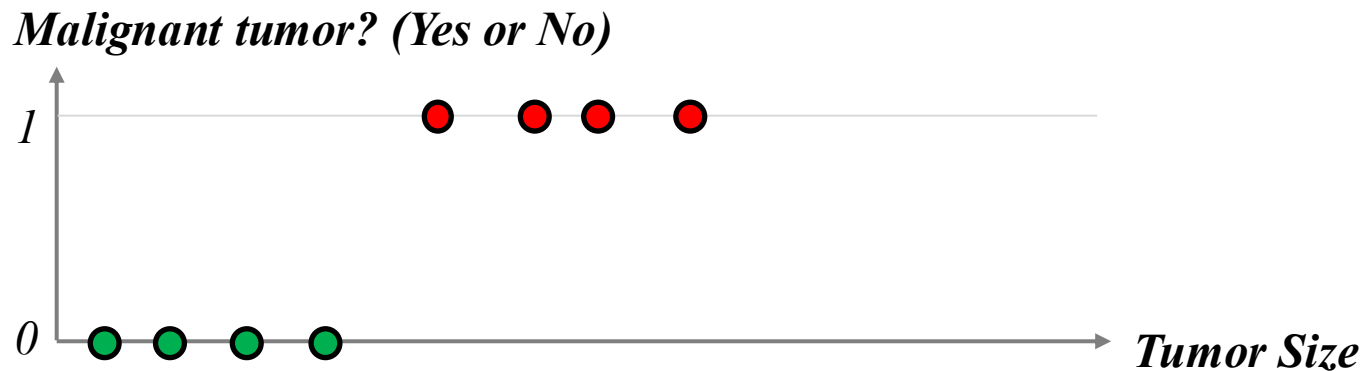


Machine Learning

Logistic Regression

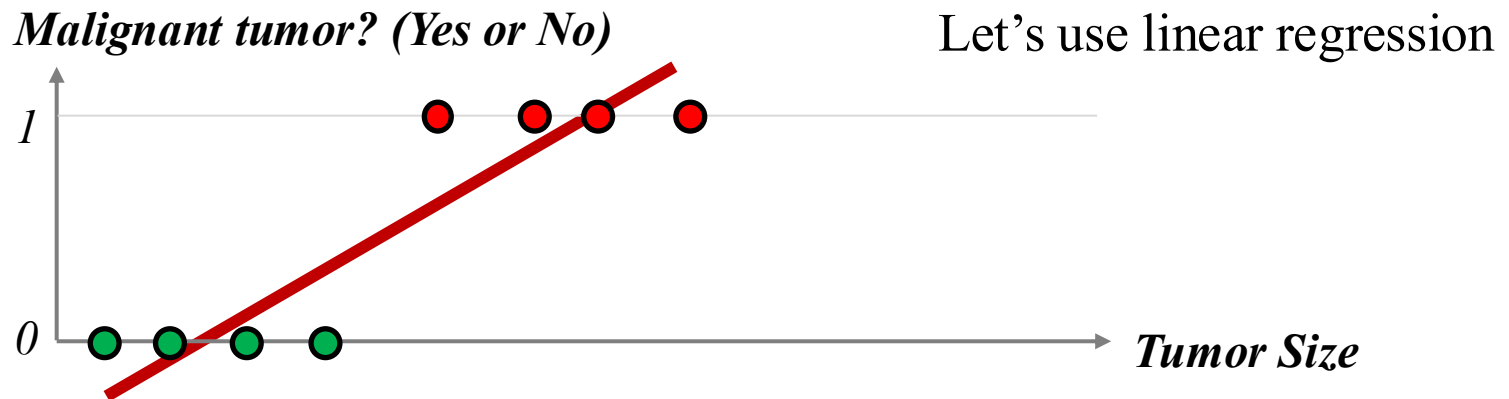


Classifying with a Regression



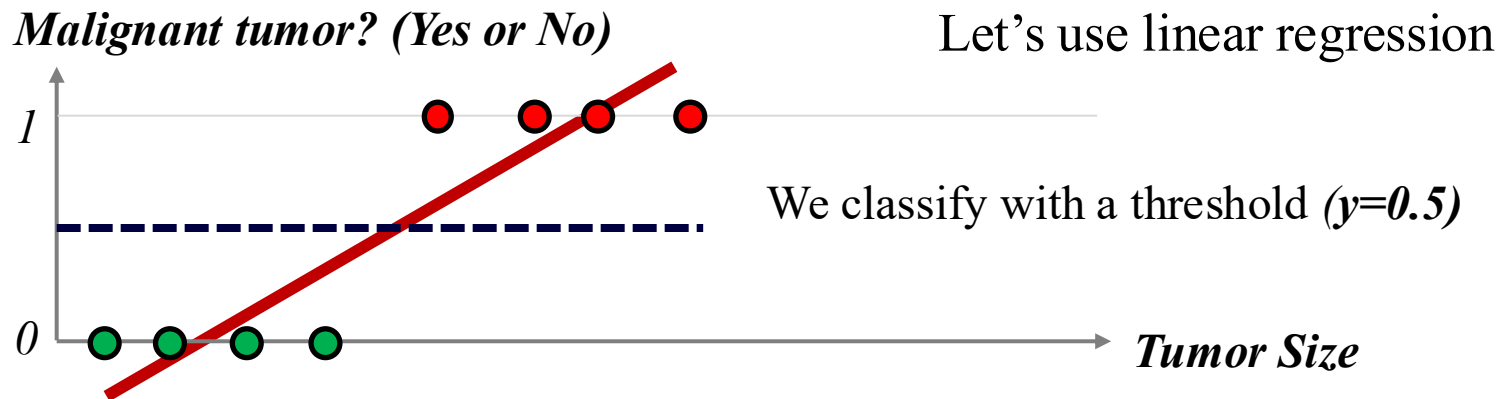
*Classifying a tumor as **benign** ($y=0$) or **malignant** ($y=1$) given its size*

Classifying with a Regression



*Classifying a tumor as **benign** ($y=0$) or **malignant** ($y=1$) given its size*

Classifying with a Regression

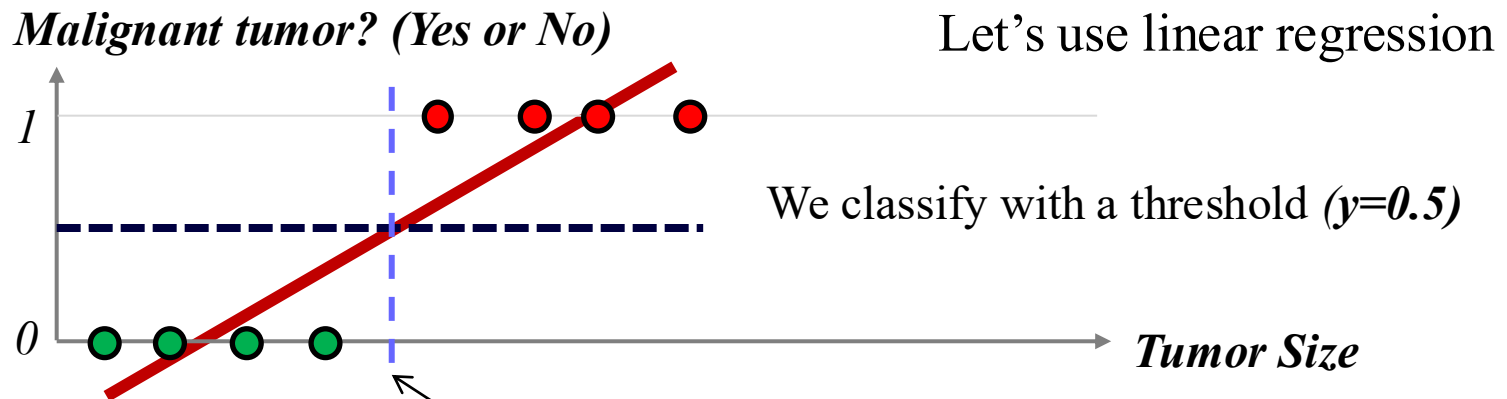


Threshold classifier output $h_{\theta}(x)$ at 0.5:

If $h_{\theta}(x) \geq 0.5$, predict “ $y = 1$ ”

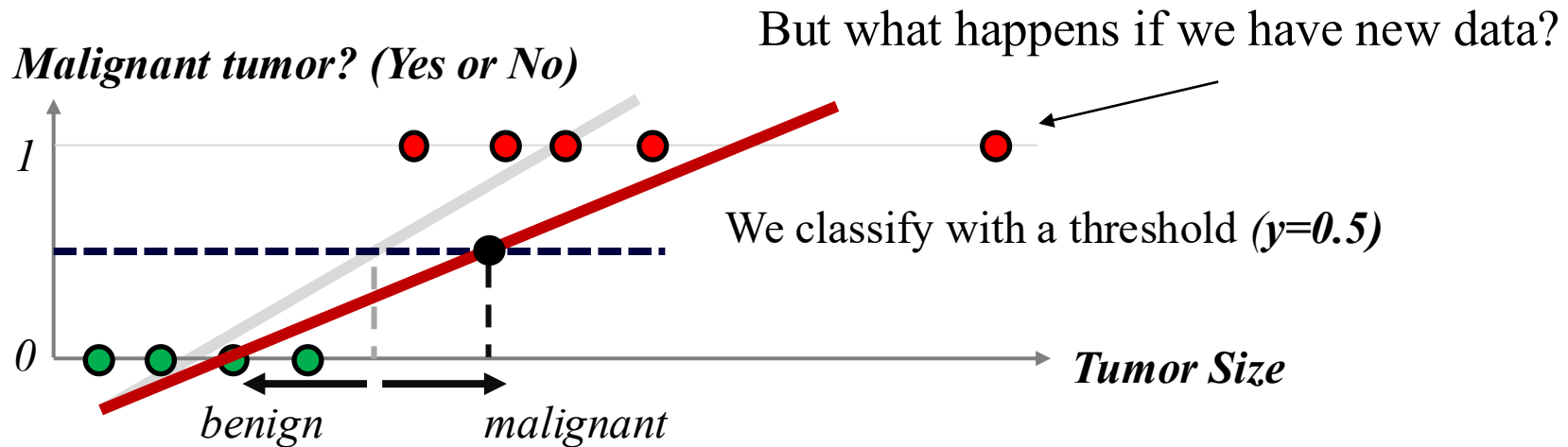
If $h_{\theta}(x) < 0.5$, predict “ $y = 0$ ”

Classifying with a Regression



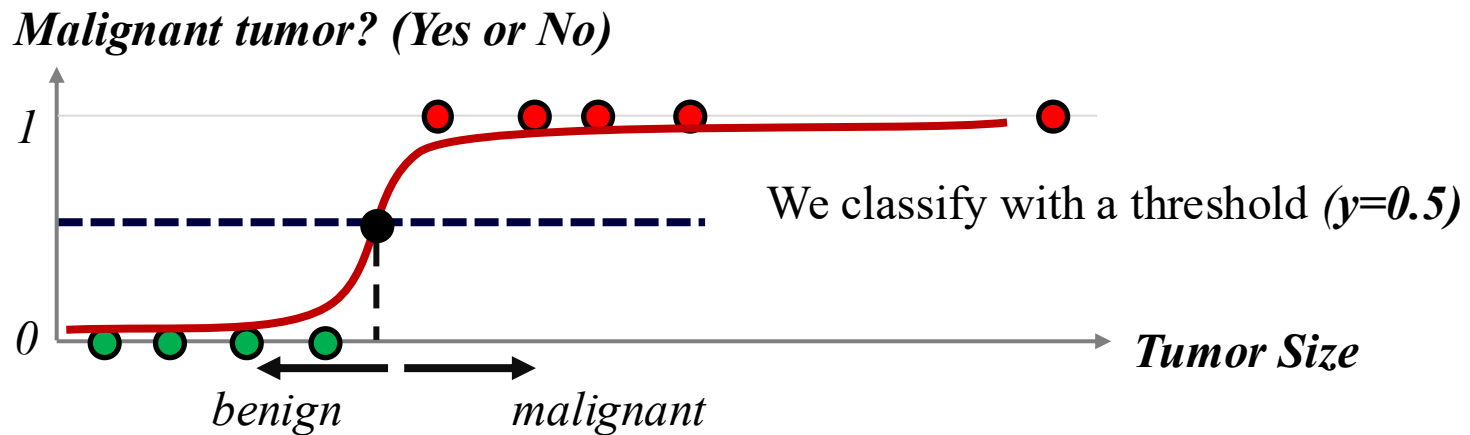
If the size is smaller than this value then it is benign otherwise it is malignant.

Classifying with a Regression



Linear regression adjusts to the new data, but the new threshold is wrong. Can we find a better curve?

Classifying with a Regression



We can use a Logistic/Sigmoid function instead of a linear function

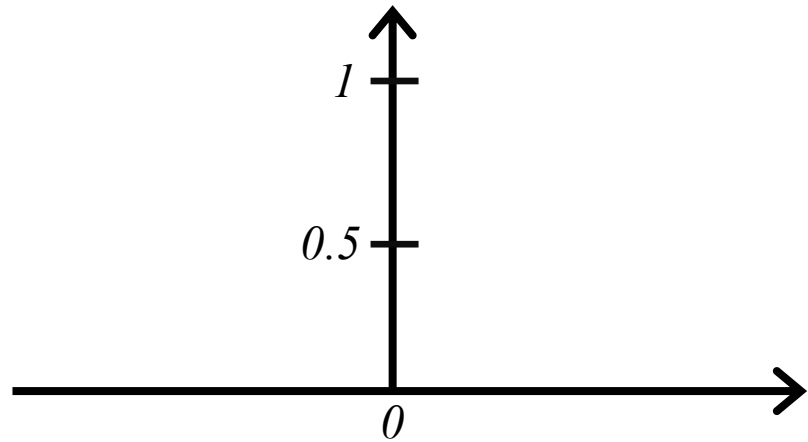
More robust to outliers
Naturally bounded between 0 and 1

Logistic Regression Model

Want $0 \leq h_{\theta}(x) \leq 1$

$$h_{\theta}(x) = \theta^T x$$

Instead of a linear function



Logistic Regression Model

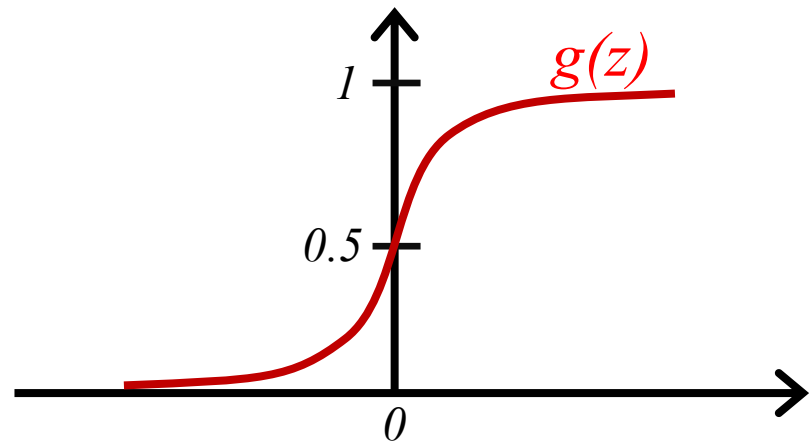
Want $0 \leq h_{\theta}(x) \leq 1$

$$h_{\theta}(x) = g(\underbrace{\theta^T x}_z)$$

We use a logistic function

$$g(z) = \frac{1}{1 + e^{-z}}$$

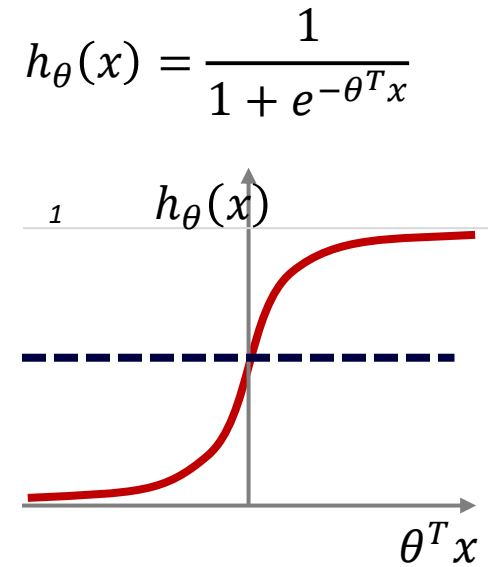
Sigmoid function
Logistic function



Logistic Regression

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$



Will predict “ $y = 1$ ” if $h_{\theta}(x) \geq 0.5$

$$g(z) \geq 0.5 \text{ when } z \geq 0$$

$$h_{\theta}(x) \geq 0.5 \text{ if } \theta^T x \geq 0$$

predict “ $y = 0$ ” if $h_{\theta}(x) < 0.5$

$$g(z) < 0.5 \text{ when } z < 0$$

$$h_{\theta}(x) < 0.5 \text{ if } \theta^T x < 0$$

Interpretation of Hypothesis Output

$h_{\theta}(x)$ = estimated probability that $y = 1$ on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$$h_{\theta}(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant $P(y = 1|x; \theta)$

“probability that $y = 1$, given x ,
parameterized by θ ”

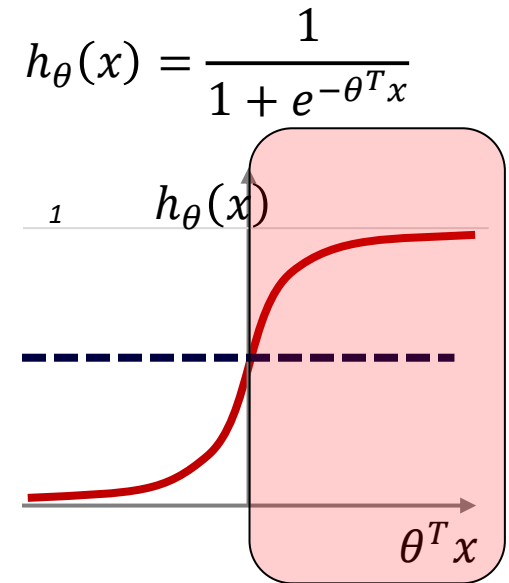
$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$

$$P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta)$$

Decision Boundary

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$



Will predict “ $y = 1$ ” if $h_{\theta}(x) \geq 0.5$

$$g(z) \geq 0.5 \text{ when } z \geq 0$$

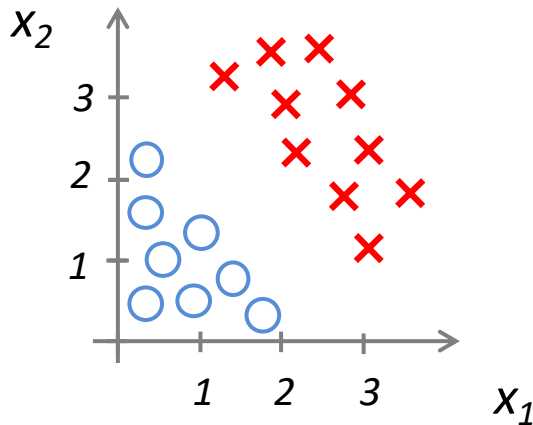
$$h_{\theta}(x) \geq 0.5 \text{ if } \theta^T x \geq 0$$

predict “ $y = 0$ ” if $h_{\theta}(x) < 0.5$

$$g(z) < 0.5 \text{ when } z < 0$$

$$h_{\theta}(x) < 0.5 \text{ if } \theta^T x < 0$$

Decision Boundary



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

A diagonal line with a positive slope, representing the decision boundary. The line passes through approximately (0, 3) and (3, 0).

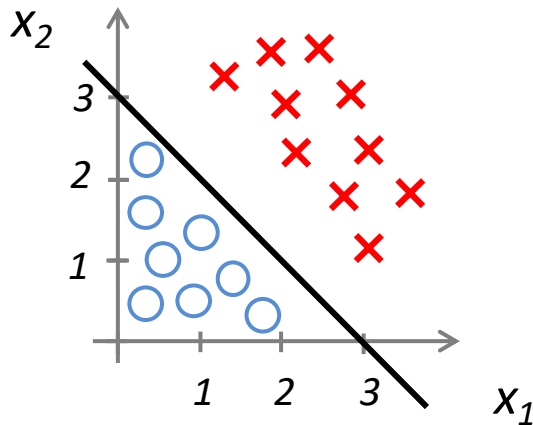
$$\Theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predict “ $y = 1$ ” if $-3 + x_1 + x_2 \geq 0$

$$x_1 + x_2 \geq 3$$

$$x_1 + x_2 < 3 \longrightarrow y = 0$$

Decision Boundary



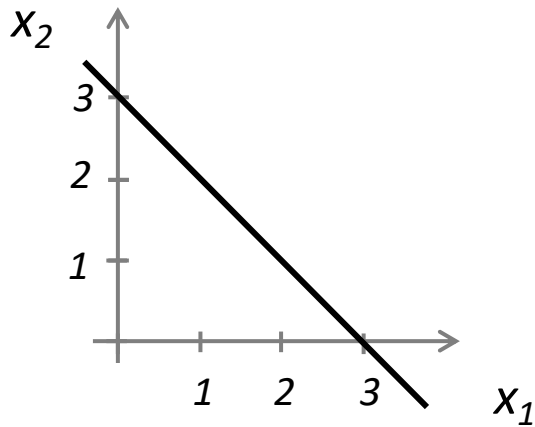
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Predict “ $y = 1$ ” if $-3 + x_1 + x_2 \geq 0$

$$x_1 + x_2 \geq 3$$

$$x_1 + x_2 < 3 \longrightarrow y = 0$$

Decision Boundary



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

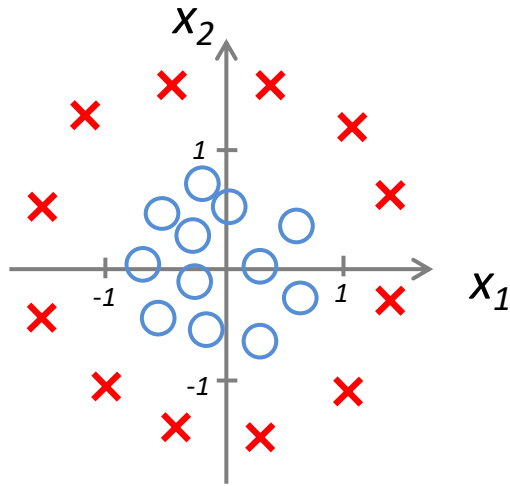
Predict “ $y = 1$ ” if $-3 + x_1 + x_2 \geq 0$

$$x_1 + x_2 \geq 3$$

$$x_1 + x_2 < 3 \longrightarrow y = 0$$

Non-linear Decision Boundaries

How can we achieve non-linear decision boundaries?



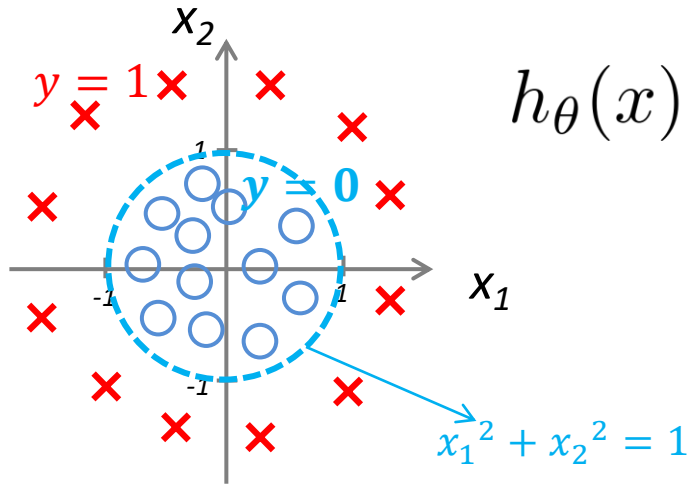
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict “ $y = 1$ ” if $-1 + x_1^2 + x_2^2 \geq 0$

Non-linear Decision Boundaries

How can we achieve non-linear decision boundaries?

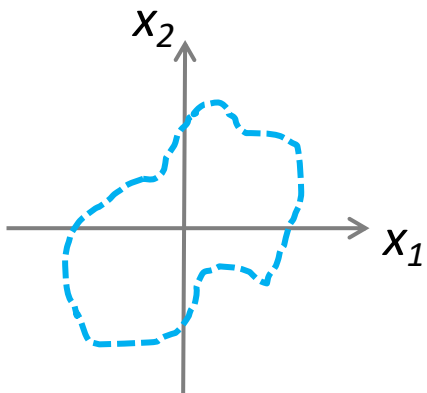


$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict “ $y = 1$ ” if $-1 + x_1^2 + x_2^2 \geq 0$

$$x_1^2 + x_2^2 \geq 1$$



In general we can achieve any decision boundaries

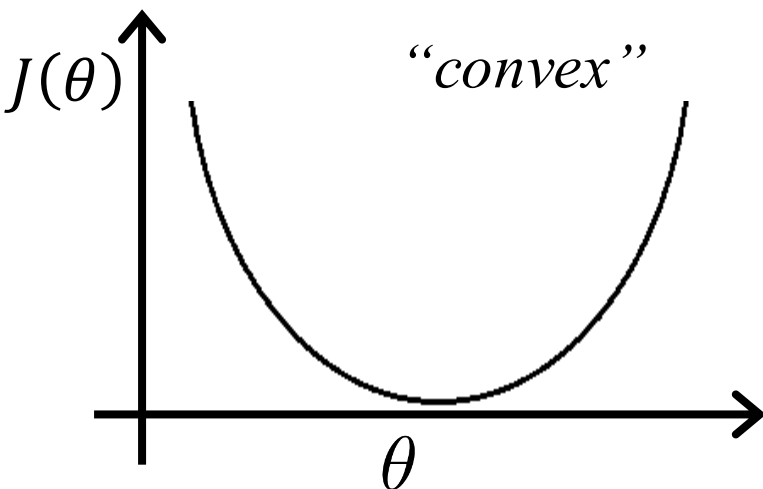
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \dots)$$

Cost Function

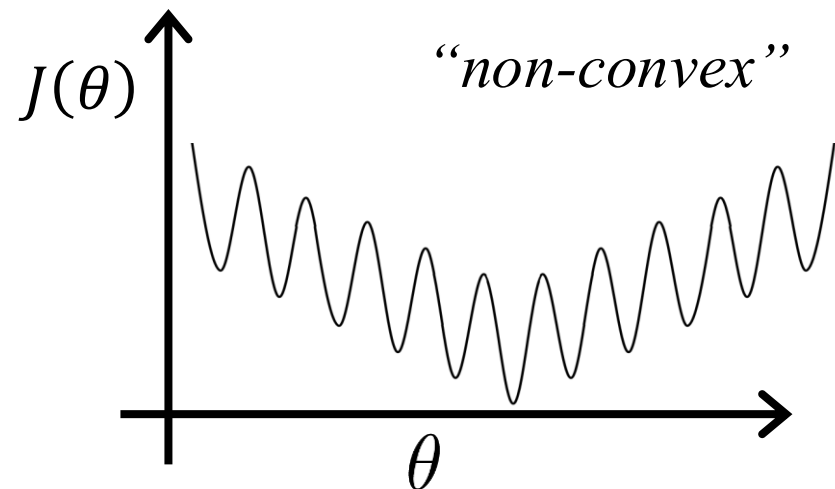
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

*From Linear
Regression*

Hypothesis function is linear
 $h_{\theta} = \theta_0 x_0 + \theta_1 x_1 + \dots + \theta_n x_n$



Hypothesis function is not linear
 $h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$



Cost Function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2 \xrightarrow{\text{Error/Loss/Penalty}} \mathcal{L}(h_{\theta}(x), y) = \frac{(h_{\theta}(x) - y)^2}{2}$$

Now instead we will measure the error/loss as:

$$\mathcal{L}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

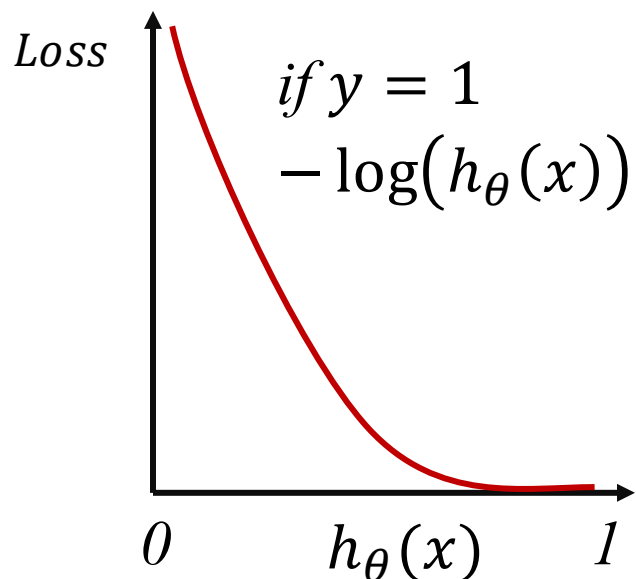
$$h_{\theta}(x) \geq 0.5 \text{ if } y = 1$$

$$h_{\theta}(x) < 0.5 \text{ if } y = 0$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Cost Function

$$Loss(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



Loss = 0 if $y = 1, h_{\theta}(x) = 1$

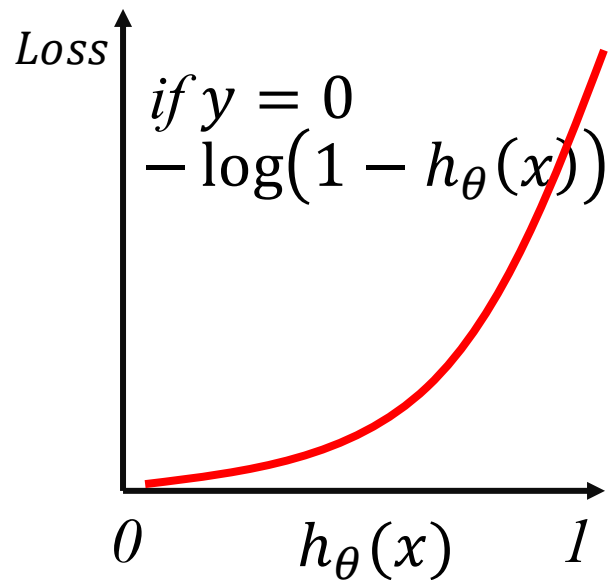
But as $h_{\theta}(x) \rightarrow 0$
 $Cost \rightarrow \infty$

Captures intuition that if $h_{\theta}(x) = 0$,
(predict $P(y = 1|x; \theta) = 0$), but $y = 1$,
we'll penalize learning algorithm by a very
large cost.

Notice that we refer as *Loss* (or
error) when it is only one training
example and as *Cost* when we take
the entire training set into account

Cost Function

$$Loss(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



Similar idea if $y=0$ and $h_{\theta}(x)=0$

Cost Function

Error/Loss/Penalty

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \longrightarrow \mathcal{L}(h_{\theta}(x), y) = \frac{(h_{\theta}(x) - y)^2}{2}$$

Now instead we will measure the error as:

$$\text{Loss}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

$$\text{Loss}(h_{\theta}(x), y) = \underbrace{-y \log(h_{\theta}(x))}_{y = 1} - \underbrace{(1 - y) \log(1 - h_{\theta}(x))}_{y = 0}$$

Logistic Regression Cost Function

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

To fit parameters θ :

$$\min_{\theta} J(\theta)$$

To make a prediction given new x :

$$\text{Output } h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

(simultaneously update all θ_j)

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

}

(simultaneously update all θ_j)

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Confusion Matrix

A confusion matrix is a table that is often used to **describe the performance of a classification model** on a set of test data for which the true values are known.

n=165	Predicted: NO	Predicted: YES
	Actual: NO	Actual: YES
	50	10
	5	100

An example confusion matrix for a binary classifier

Confusion Matrix

n=165	Predicted: NO	Predicted: YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

- **true positives (TP):** These are cases in which we **correctly** predicted **yes**.
- **true negatives (TN):** These are cases in which we **correctly** predicted **no**.
- **false positives (FP):** We predicted **yes**, but they correct prediction is **no**.
- **false negatives (FN):** We predicted **no**, but they correct prediction is **yes**.

What can we learn from this matrix?

- *There are two possible predicted classes: "yes" and "no".*
- *The classifier made a total of 165 predictions.*
- *Out of those 165 cases, the classifier predicted "yes" 110 times, and "no" 55 times.*
- *In reality, 105 examples were "yes", and 60 examples were "no".*

Confusion Matrix

		Predicted condition	
		Predicted positive	Predicted negative
Actual condition	Total population = P + N		
	Positive (P)	True positive (TP)	False negative (FN) Type Error II
	Negative (N)	False positive (FP) Type Error I	True negative (TN)

Confusion Matrix

		Predicted condition	
		Predicted positive	Predicted negative
Actual condition	Total population = P + N		
	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Confusion Matrix

		Predicted condition	
		Predicted positive	Predicted negative
Actual condition	Total population = P + N		
	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Ratios

True positive rate (TPR), Recall, Sensitivity, probability of detection $= \frac{\sum \text{True positive}}{\sum \text{Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm $= \frac{\sum \text{False positive}}{\sum \text{Condition negative}}$
False negative rate (FNR), Miss rate $= \frac{\sum \text{False negative}}{\sum \text{Condition positive}}$	Specificity (SPC), Selectivity, True negative rate (TNR) $= \frac{\sum \text{True negative}}{\sum \text{Condition negative}}$

Confusion Matrix

		Predicted condition	
		Predicted positive	Predicted negative
Actual condition	Total population = P + N		
	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Overall

Prevalence = $\frac{\sum \text{Condition positive}}{\sum \text{Total population}}$	Accuracy (ACC) = $\frac{\sum \text{True positive} + \sum \text{True negative}}{\sum \text{Total population}}$
Positive predictive value (PPV), Precision = $\frac{\sum \text{True positive}}{\sum \text{Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\sum \text{False positive}}{\sum \text{Predicted condition positive}}$



Confusion Matrix

We are trying to build a classifier to identify whether a particular insurance claim is fraudulent (or any other fraudulent activity). What do recall and precision mean in this context, and which is a better metric for our problem? Why?



Confusion Matrix

We are trying to build a classifier to identify whether a particular insurance claim is fraudulent (or any other fraudulent activity). What do recall and precision mean in this context, and which is a better metric for our problem? Why?

- **Precision** is the fraction of cases that are fraudulent out of those that our classifier labelled as positive. The problem with a low precision is that our investigators will spend a lot of time investigating claims that are actually legitimate.
- **Recall** is the fraction of fraudulent cases our classifier finds. The problem with a low recall is that we would be paying out on a lot of undetected fraudulent claims.

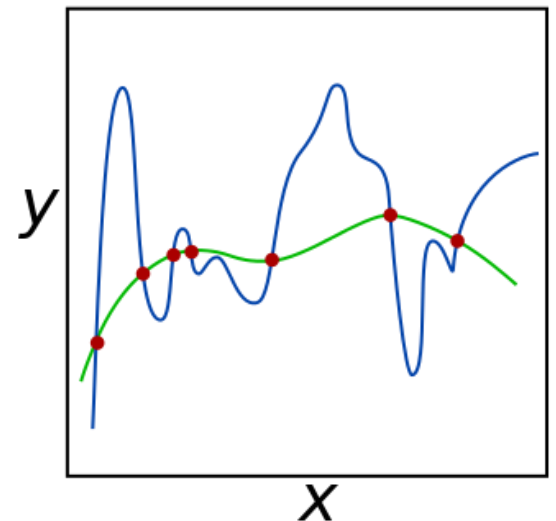
Confusion Matrix

		Predicted condition	
		Predicted positive	Predicted negative
Actual condition	Total population = P + N		
	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)
Overall		Prevalence = $\frac{\sum \text{Condition positive}}{\sum \text{Total population}}$	Accuracy (ACC) = $\frac{\sum \text{True positive} + \sum \text{True negative}}{\sum \text{Total population}}$
		Positive predictive value (PPV), Precision = $\frac{\sum \text{True positive}}{\sum \text{Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\sum \text{False positive}}{\sum \text{Predicted condition positive}}$

Null Error Rate: This is how often you would be wrong if you always predicted the majority class.

Machine Learning

Multiclass classification



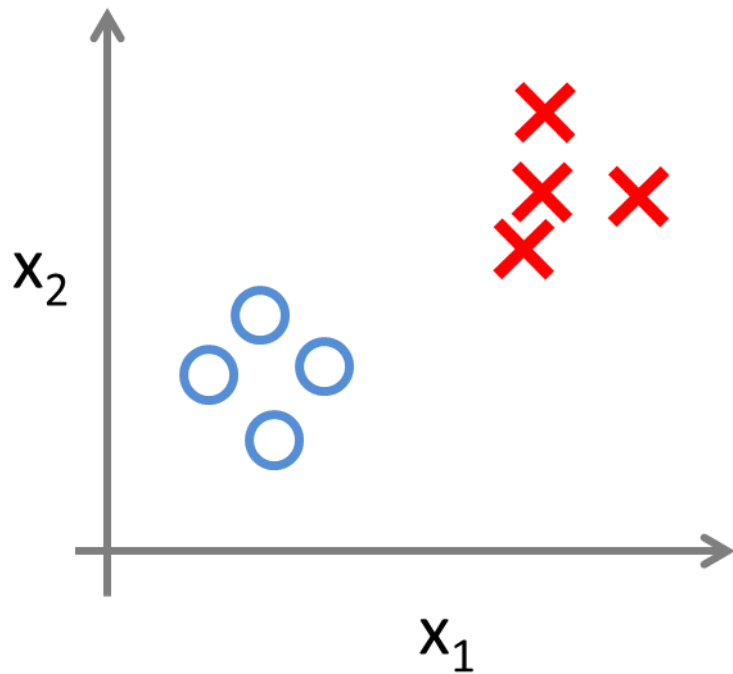


Multiclass Classification

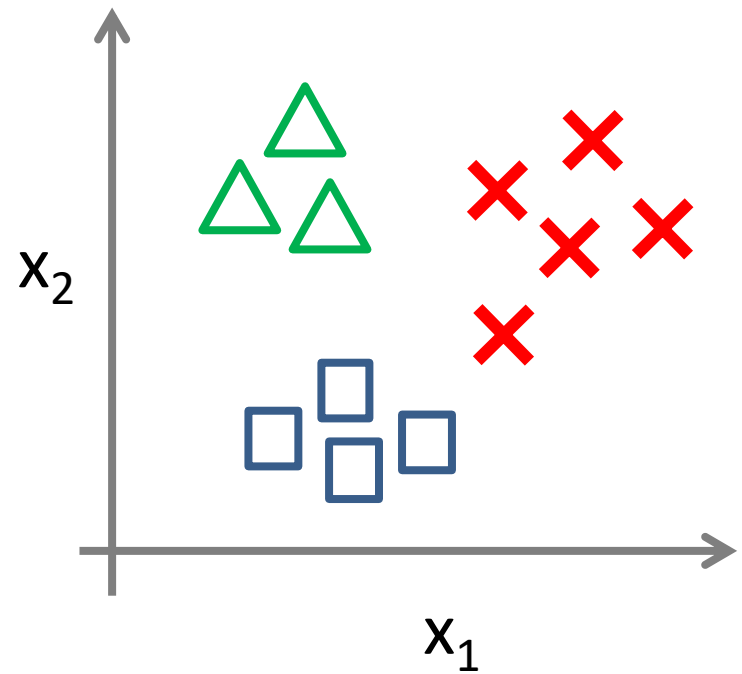
- Email tagging: Work, Friends, Family, Hobby
- Medical diagrams: Not ill, Cold, Flu
- Weather: Sunny, Cloudy, Rain, Snow
- Image recognition: Dog, Cat, Person

Multiclass Classification

Binary classification:

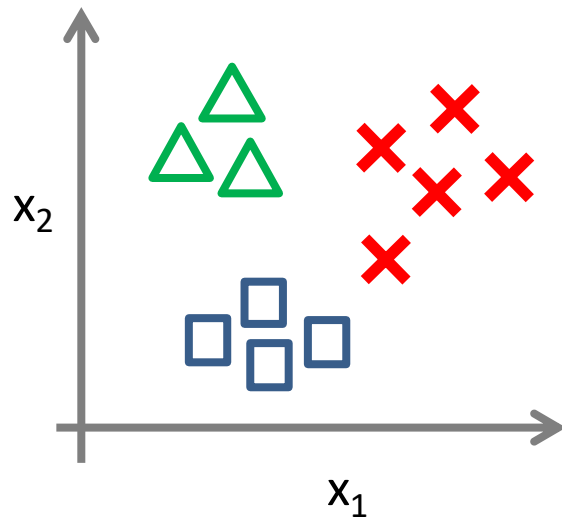


Multi-class classification:



Multiclass Classification

One-vs-all (one-vs-rest)

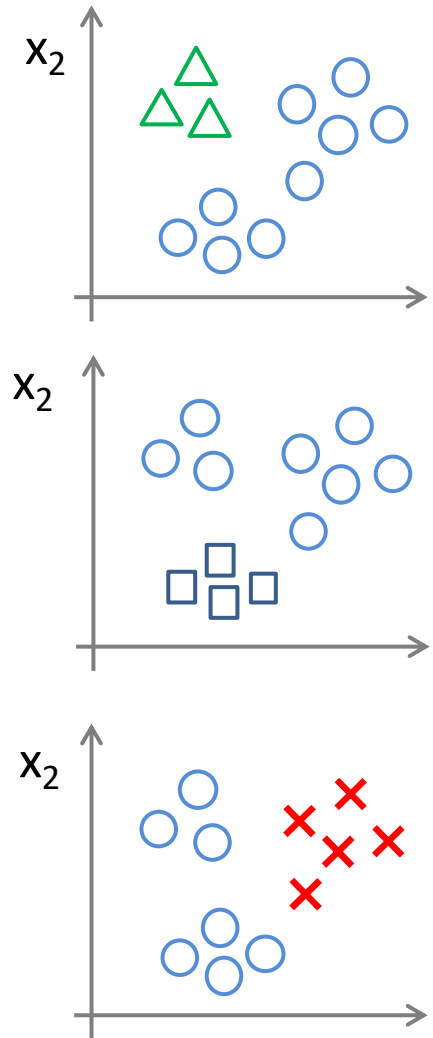


Class 1: 

Class 2: 

Class 3: 

$$h_{\theta}^{(i)}(x) = P(y = i | x; \theta) \quad (i = 1, 2, 3)$$



Multiclass Classification

One-vs-all (one-vs-rest)

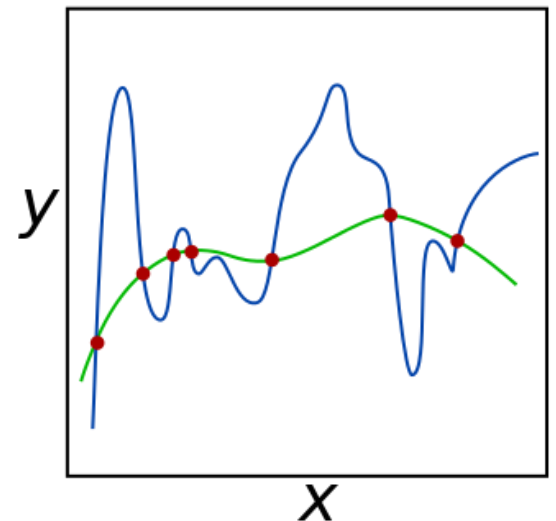
Train a logistic regression classifier $h_{\theta}^{(i)}(x)$ for each class i to predict the probability that $y = i$.

On a new input x , to make a prediction, pick the class i that maximizes

$$\max_i h_{\theta}^{(i)}(x)$$

Machine Learning

Regularization



Logistic Regression Model

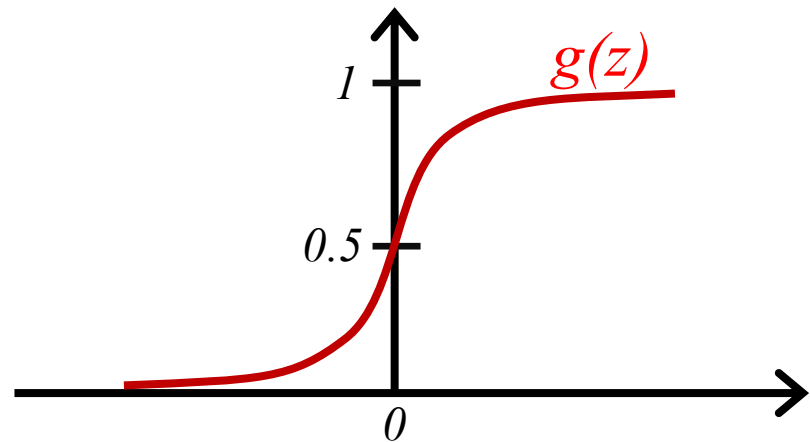
Want $0 \leq h_{\theta}(x) \leq 1$

$$h_{\theta}(x) = g(\underbrace{\theta^T x}_z)$$

We use a logistic function

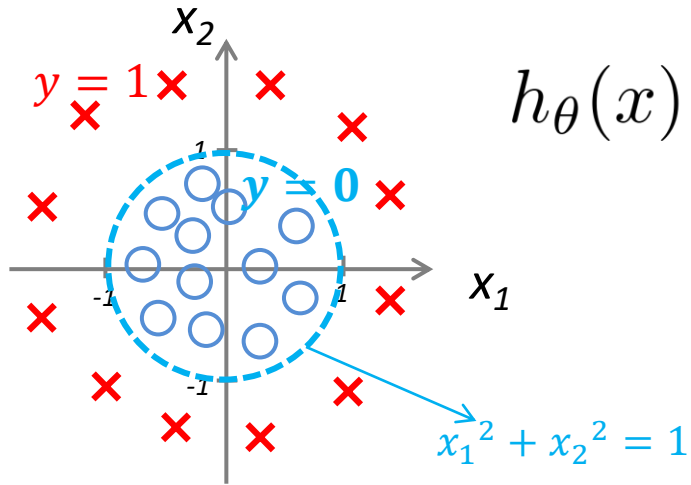
$$g(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid function
Logistic function



Non-linear Decision Boundaries

How can we achieve non-linear decision boundaries?

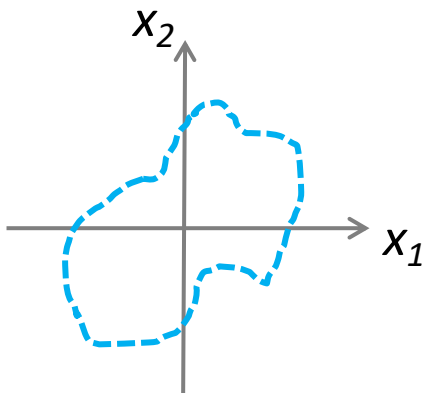


$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict “ $y = 1$ ” if $-1 + x_1^2 + x_2^2 \geq 0$

$$x_1^2 + x_2^2 \geq 1$$

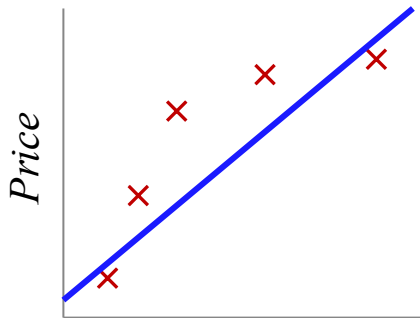


In general we can achieve any decision boundaries

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \dots)$$

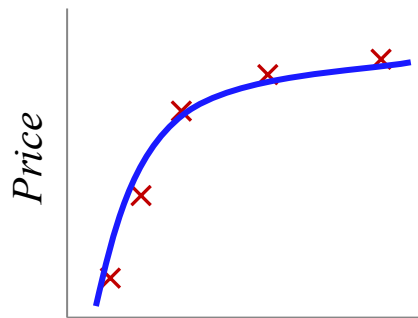
Overfitting

Example: Linear regression (housing prices)

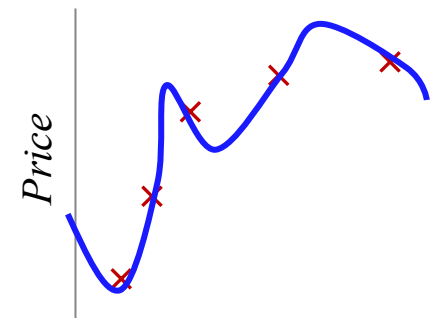


$$\theta_0 + \theta_1 x$$

underfit or high bias



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



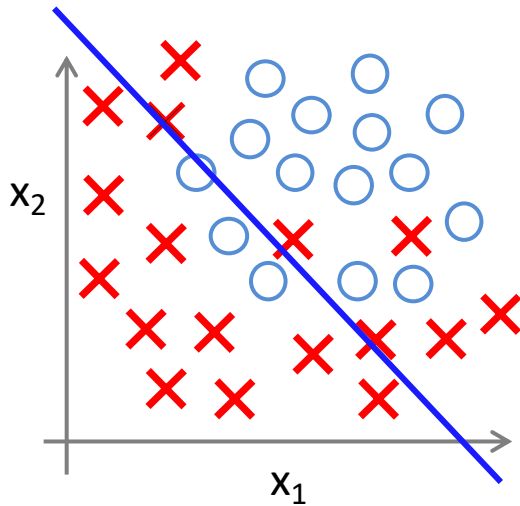
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

overfit or high variance

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

Overfitting

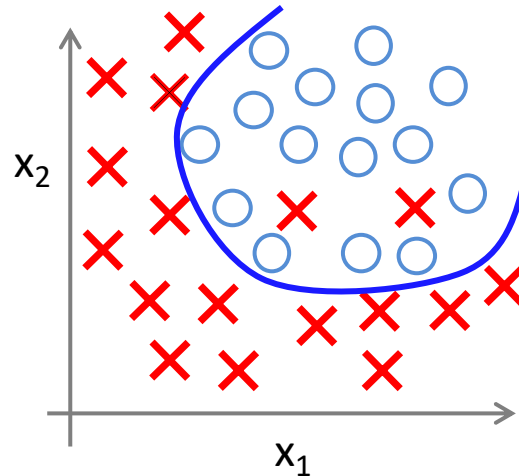
Example: Logistic regression



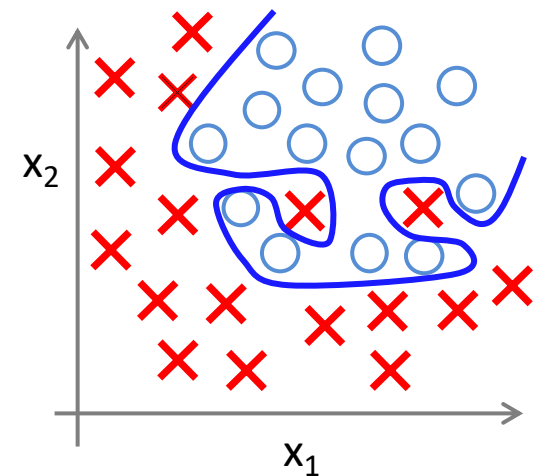
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

(g = sigmoid function)

underfit or high bias



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

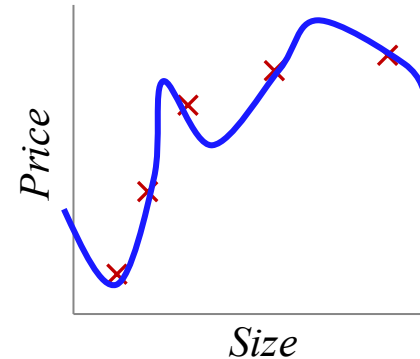


$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

overfit or high variance

Addressing overfitting:

x_1 = size of house
 x_2 = no. of bedrooms
 x_3 = no. of floors
 x_4 = age of house
 x_5 =
 x_6 = kitchen size
:
 x_{100}

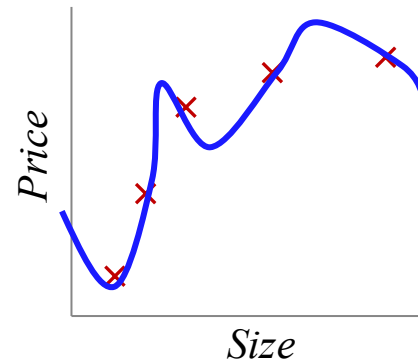


Visualizing higher dimensions is difficult

- If we have many features it may not be obvious which features we should drop to avoid overfitting.
- It may also be the case that every feature provides useful information for predicting the price.

Addressing overfitting:

x_1 = size of house
 x_2 = no. of bedrooms
 x_3 = no. of floors
 x_4 = age of house
 x_5 =
 x_6 = kitchen size
...

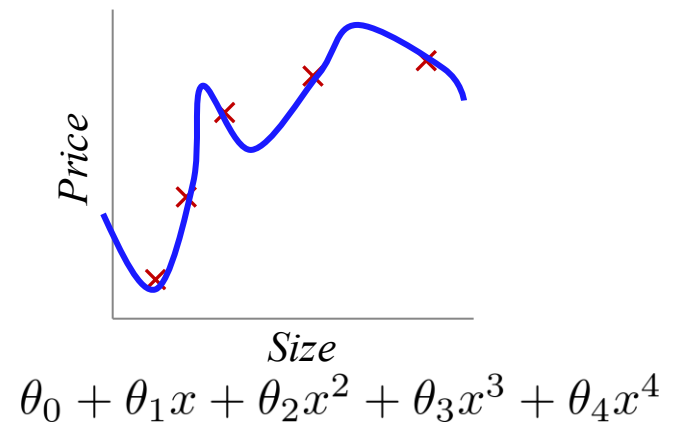
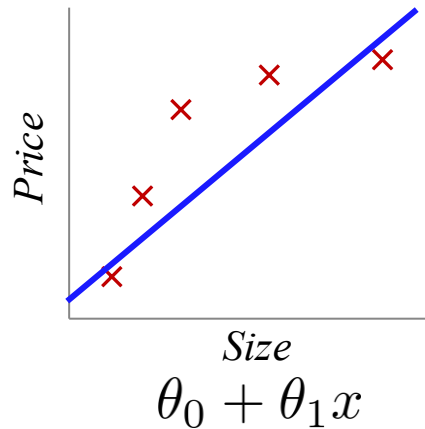


Visualizing higher dimensions is difficult

Options:

- Reduce number of features.
 - Manually select which features to keep.
 - Feature reduction algorithm (later in course).
- Regularization.
 - Keep all the features, but reduce magnitude/values of parameters θ_j .
 - Works well when we have a lot of features, each of which contributes a bit to predicting y .

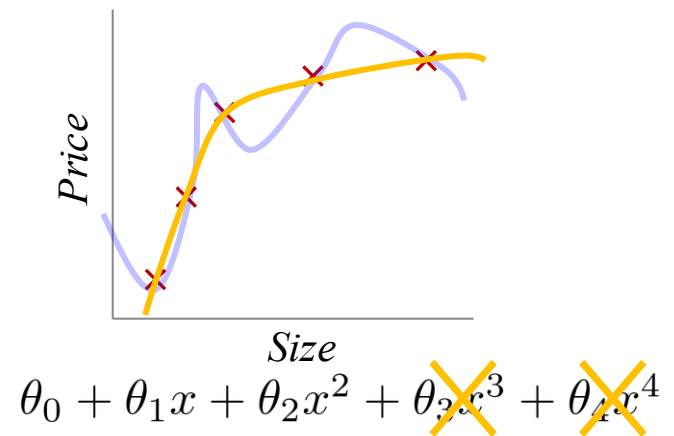
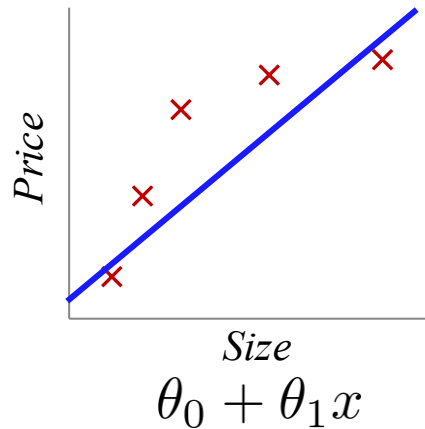
Regularization Intuition



Suppose we penalize and make θ_3 and θ_4 really small.

$$\min_{\theta} \underbrace{\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\theta_3 \approx 0} + \underbrace{1000 \theta_3^2 + 1000 \theta_4^2}_{\theta_4 \approx 0}$$

Regularization Intuition



Suppose we penalize and make θ_3 and θ_4 really small.

$$\min_{\theta} \underbrace{\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\theta_3 \approx 0} + \underbrace{1000 \theta_3^2 + 1000 \theta_4^2}_{\theta_4 \approx 0}$$

Regularization

- Small values for parameters $\theta_0, \theta_1, \dots, \theta_n$
 - “Simpler” hypothesis
 - Less prone to overfitting
- Housing:
 - Features: $x_1, x_2, \dots, x_{100}$
 - Features: $\theta_0, \theta_1, \theta_2, \dots, \theta_{100}$

Similar to the idea of penalizing θ_3 and θ_4 but applied to all features.

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

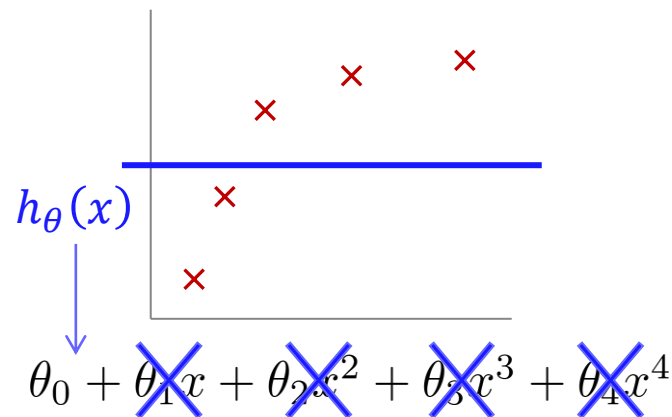
Regularization

In regularized linear regression, we choose θ to minimize:

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

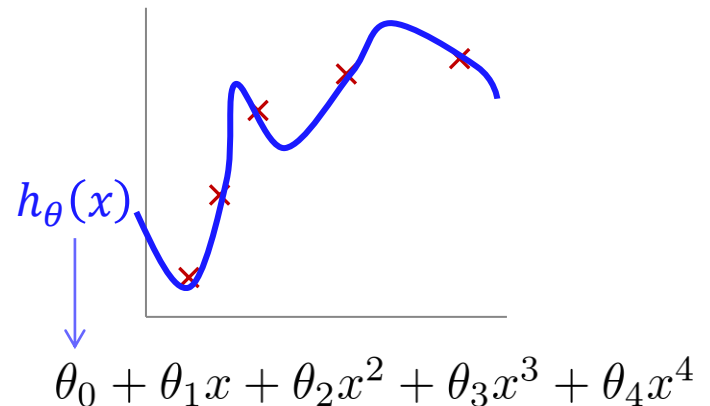
Parameter λ balances the effect of regularization:

If λ is too large (10^{10})



underfit or high bias

If λ is too low (10^{-10})



overfit or high variance

Gradient Descent with Regularization

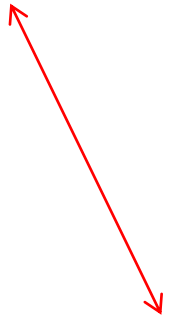
Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

($j = \text{~~x~~, 1, 2, 3, \dots, n$)

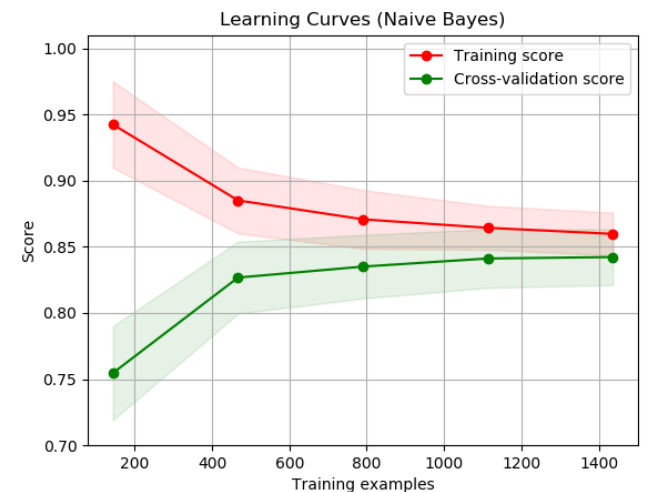
}


$$\theta_j := \underbrace{\theta_j (1 - \alpha \frac{\lambda}{m})}_{< 1} - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

< 1 , this term shrinks the parameters

Machine Learning

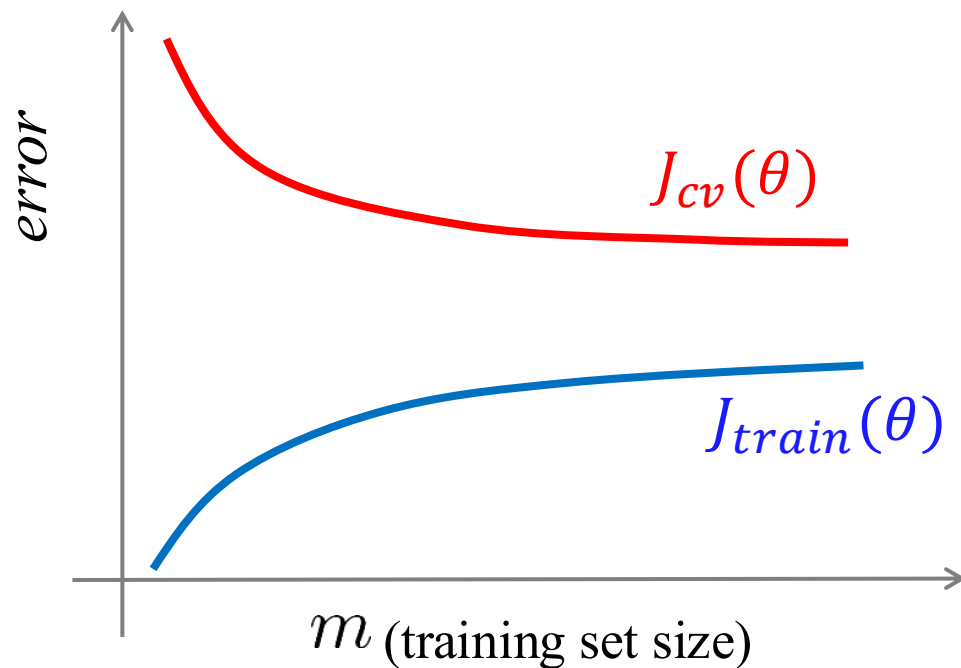
Overfitting and Learning Curves



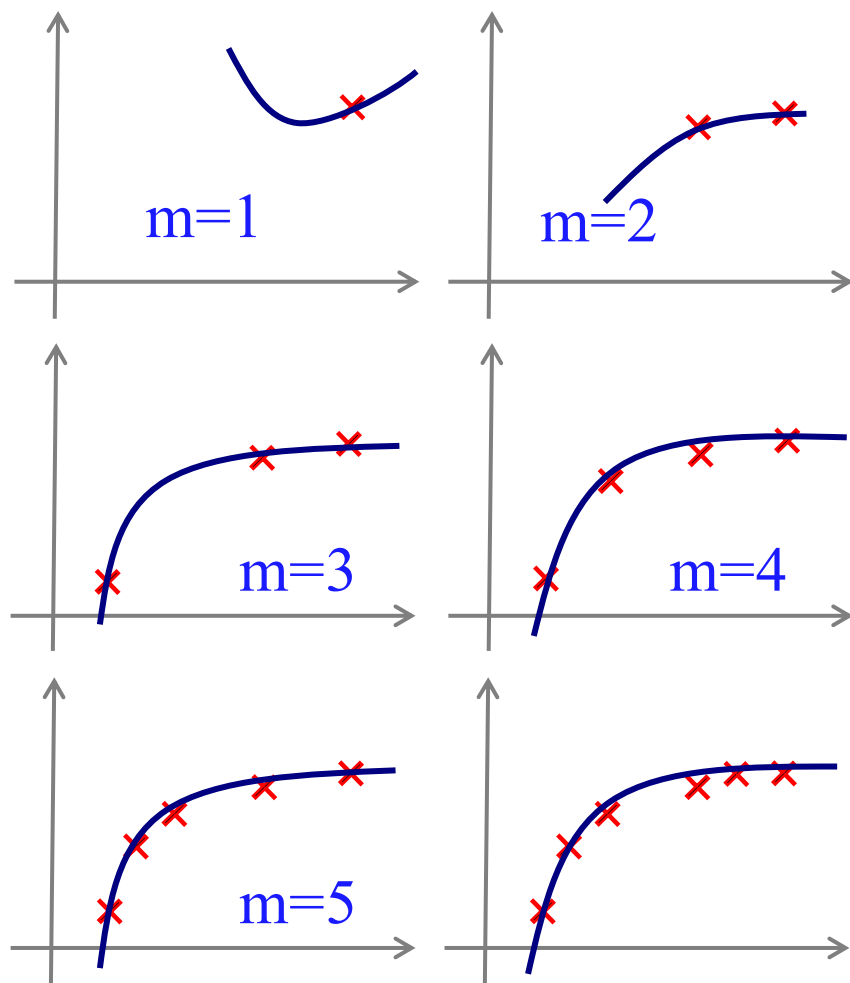
Learning curves

$$J_{train}(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$J_{cv}(\theta) = \frac{1}{2m_{cv}} \sum_{i=1}^{m_{cv}} (h_{\theta}(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

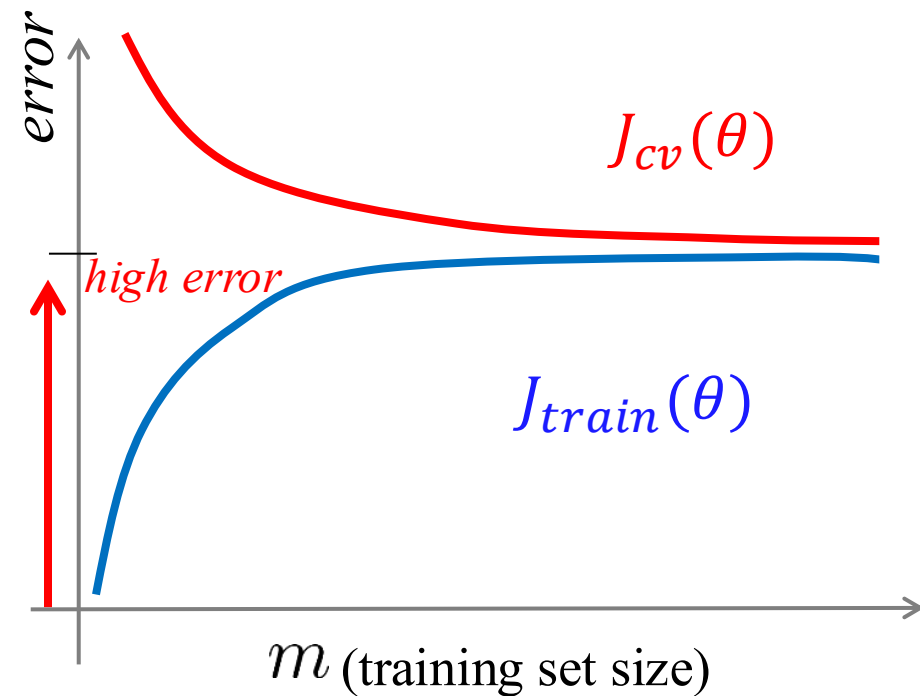


$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

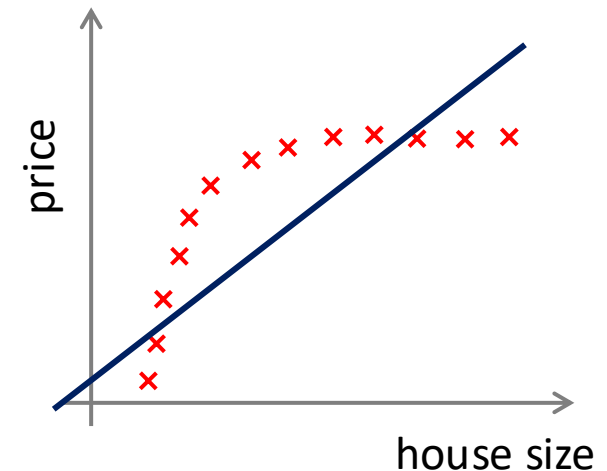
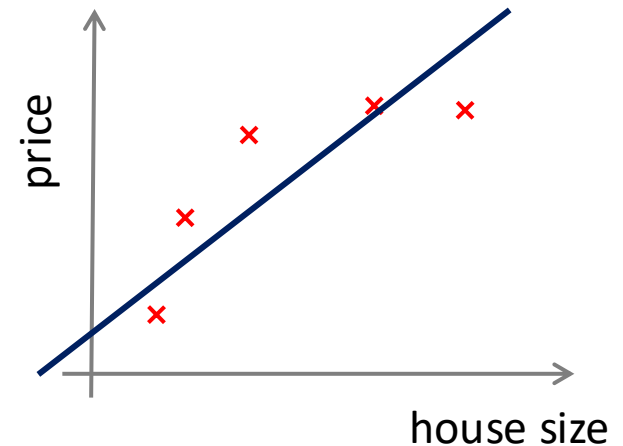


High Bias

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

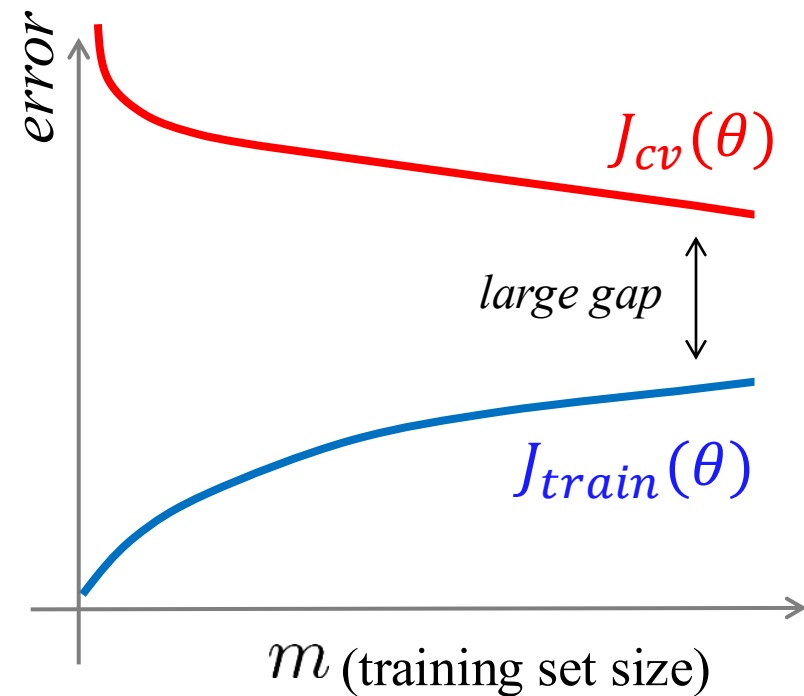


If a learning algorithm is suffering from high bias, getting more training data won't help much.

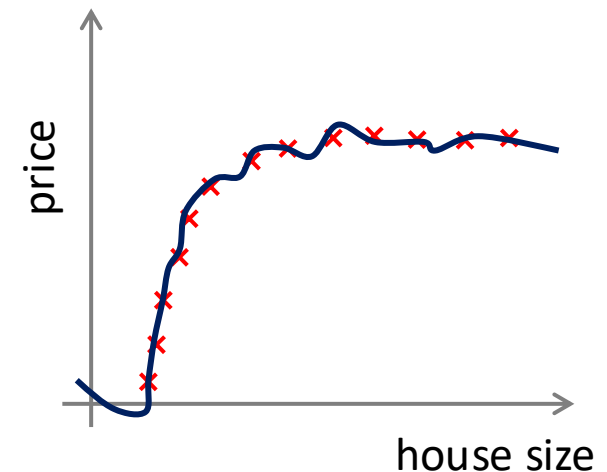
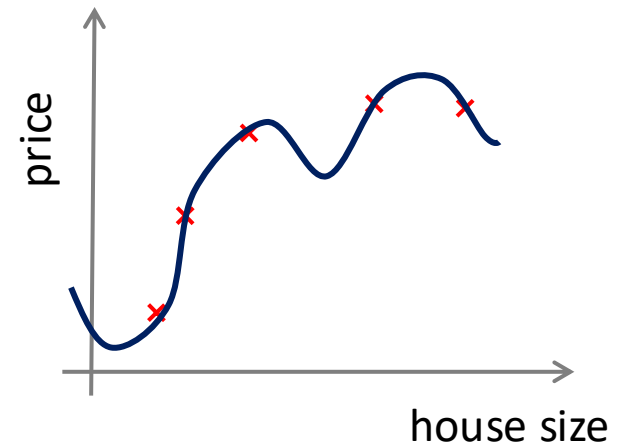


High Variance

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \dots + \theta_{100} x^{100}$$



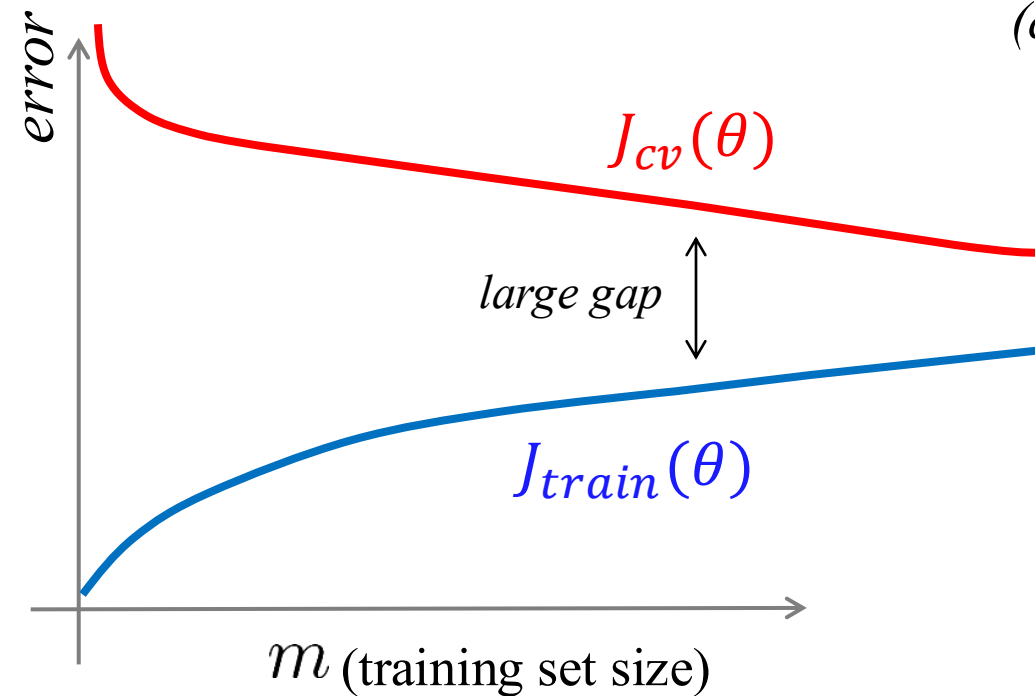
If a learning algorithm is suffering from high variance, getting more training data is likely to help.



High Variance

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \dots + \theta_{100} x^{100}$$

(change the model or add regularization)



If a learning algorithm is suffering from high variance, getting more training data is likely to help.

